Week 3 Worked Companion

Using the worked companion

Attempt the student exercises independently first. This companion contains hints and complete solutions. Read only as far as needed, then return to your attempt.

Examples are original teaching examples. Follow each official link for the source problem and its complete contract.

Student exercises

Instructor guide

Download all code

74 · Staged hints

LeetCode 74: Search a 2D Matrix · Medium

New exercise

H1. Which row and column correspond to flat index 4?

H2. Is the flattened order sorted without the cross-row condition?

H3. Keep a closed candidate interval [left,right]; read mid at row mid//C and column mid%C.

74 · Correctness and cost

The cross-row property makes the flattened order sorted. Comparisons eliminate an impossible half; every update shrinks the interval, and an empty interval means absence.

R,C are row/column counts. O(log(RC)) time, O(1) auxiliary space; no flattened copy.

74 · Counterexamples and checks

left=mid can stall on two candidates. [[1,9],[2,10]] has sorted rows but violates this contract.

```
matrix=[[1,4,7],[9,12,18]], target=12 → True
```

Trace before execution: write down each state and the return value, then compare with the code. Follow-up: what changes if mutation is forbidden, a different output is requested, or structure costs change?

74 · Implementation 1/1

The complete implementation is also in the code bundle.

```
def search_matrix(matrix, target):
    rows, cols = len(matrix), len(matrix[0])
    left, right = 0, rows * cols - 1
    while left <= right:
        mid = (left + right) // 2
        value = matrix[mid // cols][mid % cols]
        if value == target:
            return True
        if value < target:
            left = mid + 1
        else:
            right = mid - 1
    return False
```

33 · Staged hints

LeetCode 33: Search in Rotated Sorted Array · Medium

Extension

H1. Which side of mid must be sorted?

H2. Does target lie inside that sorted value range?

H3. Identify a sorted half and retain it only when its value range contains target.

33 · Correctness and cost

Distinct values and one rotation ensure a sorted half. Range tests discard only an impossible half; mid±1 guarantees progress.

n is length. O(log n) time, O(1) auxiliary space.

33 · Counterexamples and checks

With duplicates, [1,0,1,1,1] makes the sorted-side test ambiguous; duplicates are outside this contract.

```
nums=[6,8,1,3,5], target=3 → 3
```

Trace before execution: write down each state and the return value, then compare with the code. Follow-up: what changes if mutation is forbidden, a different output is requested, or structure costs change?

33 · Implementation 1/1

The complete implementation is also in the code bundle.

```
def search_rotated(nums, target):
    left, right = 0, len(nums) - 1
    while left <= right:
        mid = (left + right) // 2
        if nums[mid] == target:
            return mid
        if nums[left] <= nums[mid]:
            if nums[left] <= target < nums[mid]:
                right = mid - 1
            else:
                left = mid + 1
        else:
            if nums[mid] < target <= nums[right]:
                left = mid + 1
            else:
                right = mid - 1
    return -1
```

206 · Staged hints

LeetCode 206: Reverse Linked List · Easy

New exercise

H1. How will the remaining chain be found after next is overwritten?

H2. What are the heads of the processed and unprocessed parts?

H3. Save following, redirect current.next to previous, then advance both references.

206 · Correctness and cost

previous holds the reversed prefix and current the untouched suffix. Moving one node preserves all nodes; when current is None, previous is the full reversed list.

n is node count. O(n) time, O(1) auxiliary space; nodes are reused.

206 · Counterexamples and checks

Overwriting without saving next loses the suffix. Two nodes with equal values are still different objects.

```
head=[4,6,9] → [9,6,4]
```

Trace before execution: write down each state and the return value, then compare with the code. Follow-up: what changes if mutation is forbidden, a different output is requested, or structure costs change?

206 · Implementation 1/1

The complete implementation is also in the code bundle.

```
def reverse_list(head):
    previous, current = None, head
    while current:
        following = current.next
        current.next = previous
        previous, current = current, following
    return previous
```

21 · Staged hints

LeetCode 21: Merge Two Sorted Lists · Easy

New exercise

H1. Can any unprocessed value precede the smaller head?

H2. Must the first result node need special handling?

H3. Maintain tail after a dummy node; link the smaller head, then attach the remaining suffix.

21 · Correctness and cost

The prefix through tail is sorted. The smaller head is minimal among all remaining nodes; the leftover suffix is already sorted.

n,m are node counts. O(n+m) time, O(1) auxiliary space; input links change.

21 · Counterexamples and checks

Returning dummy includes a fake node. This implementation assumes acyclic input lists with no shared nodes.

```
first=[1,5], second=[2,3] → [1,2,3,5]
```

Trace before execution: write down each state and the return value, then compare with the code. Follow-up: what changes if mutation is forbidden, a different output is requested, or structure costs change?

21 · Implementation 1/1

The complete implementation is also in the code bundle.

```
def merge_lists(first, second):
    dummy = tail = ListNode()
    while first and second:
        if first.val <= second.val:
            tail.next, first = first, first.next
        else:
            tail.next, second = second, second.next
        tail = tail.next
    tail.next = first or second
    return dummy.next
```

Execution and sources

```
cd code
python verify.py
```

Common objects and imports are in structures.py and solutions.py. On LeetCode, adapt the function to the official method signature.

Erik Demaine and Srini Devadas, Introduction to Algorithms, MIT 6.006 (Fall 2011).

Short checks

1. In a four-column matrix, which cell has flat index 6?
2. Why must the next node be saved before reversing a link?
3. What does `dummy.next` return when both input lists are empty?

Answers: (1) row 1, column 2; (2) otherwise the unprocessed suffix is no longer reachable from the current node; (3) `None`.